LAPORAN PRAKTIK GIT DAN GITHUB

PENERAPAN GIT DAN GITHUB DALAM PENGELOLAAN REPOSITORY WEBSITE JELAJAH NUSANTARA

Laporan ini disusun untuk memenuhi tugas Mata Kuliah pengolahan citra digital

Dosen Pengampu : Andi Aljabar, S.Kom.,MTA.

Disusun Oleh:

Shela Rahma Fitri (24260012)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK DAN ILMU KOMPUTER

UNIVERSITAS NAHDLATUL ULAMA INDONESIA

BOGOR

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT atas rahmat dan karunia-Nya sehingga laporan praktik berjudul "Penerapan Git dan GitHub dalam Pengelolaan Repository Website Jelajah Nusantara" dapat diselesaikan. Laporan ini disusun untuk memenuhi tugas dalam mata kuliah Pengolahan Citra Digital..

Laporan ini memaparkan perjalanan praktik dari awal hingga akhir: memeriksa instalasi Git, membuat website sederhana, mengelola versinya pada repository lokal, hingga menyinkronkannya dengan GitHub melalui perintah push dan pull. Melalui kegiatan ini penulis memperoleh gambaran nyata tentang cara kerja pengelolaan versi yang sebelumnya hanya dikenal secara teori.

Penulis mengucapkan terima kasih kepada Andi Aljabar, S.Kom.,MTA. selaku dosen pengampu atas arahan dan kesempatan yang diberikan, serta kepada semua pihak yang telah membantu kelancaran praktik dan penyusunan laporan ini.

Penulis menyadari laporan ini masih memiliki kekurangan. Kritik dan saran yang membangun sangat penulis harapkan untuk perbaikan di kemudian hari. Semoga laporan ini bermanfaat bagi pembaca yang sedang mempelajari Git dan GitHub.

Bogor, Oktober 2026

Penulis,

Shela Rahma Fitri

DAFTAR ISI

KATA PENGANTAR i
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Fokus Permasalahan 1
1.3 Tujuan Praktik 2
1.4 Manfaat Praktik 2
BAB II KONSEP DASAR PENGELOLAAN VERSI 4
2.1 Pengelolaan Versi dan Alasan Penggunaannya 4
2.2 Git sebagai Sistem Pengelolaan Versi Terdistribusi 4
2.3 GitHub sebagai Layanan Hosting Repository 4
2.4 Repository, Staging Area, dan Commit 5
2.5 Branch 5
2.6 Remote, Push, dan Pull 5
2.7 Perintah Git yang Digunakan dalam Praktik 6
BAB III PERSIAPAN DAN ALUR PELAKSANAAN 7
3.1 Waktu dan Tempat Pelaksanaan 7
3.2 Perangkat dan Aplikasi 7
3.3 Objek Praktik: Website Jelajah Nusantara 7
3.4 Alur Pelaksanaan Praktik 8
BAB IV IMPLEMENTASI DAN HASIL PRAKTIK 9
4.1 Gambaran Umum Pelaksanaan 9
4.2 Penyiapan Lingkungan Kerja Git 9
4.2.1 Tahap 1: Memeriksa Instalasi Git 9
4.2.2 Tahap 2: Mengatur Identitas Pengguna Git 9
4.3 Pembuatan Website Jelajah Nusantara 10
4.3.1 Tahap 3: Menyusun Struktur Halaman dengan HTML 10
4.3.2 Tahap 4: Mengatur Tampilan dengan CSS 10
4.3.3 Tahap 5: Menguji Website melalui Peramban 10
4.4 Pengelolaan Versi pada Repository Lokal 11
4.4.1 Tahap 6: Membuat Repository Lokal dengan git init 11
4.4.2 Tahap 7: Memasukkan Berkas ke Staging Area dengan git add 11
4.4.3 Tahap 8: Menyimpan Perubahan dengan git commit 12
4.5 Penghubungan Repository Lokal dengan GitHub 12
4.5.1 Tahap 9: Membuat Repository Baru di GitHub 12
4.5.2 Tahap 10: Menghubungkan Repository Lokal dengan git remote 14

4.5.3 Tahap 11: Mengatur Branch Utama menjadi main 15
4.6 Sinkronisasi Proyek antara Komputer Lokal dan GitHub.....15
4.6.1 Tahap 12: Mengunggah Proyek dengan git push 15
4.6.2 Tahap 13: Memeriksa Hasil Pengunggahan di GitHub.....16
4.6.3 Tahap 14: Membuat dan Mengunggah README.md.....16
4.6.4 Tahap 15: Menambahkan catatan-praktik.txt Langsung melalui GitHub 17
4.6.5 Tahap 16: Mengambil Perubahan dari GitHub dengan git pull 18
4.7 Rekapitulasi dan Pembahasan Hasil.....19
BAB V PENUTUP.....22
5.1 Kesimpulan.....22
5.2 Saran.....22
DAFTAR PUSTAKA.....23

BAB I PENDAHULUAN

1.1 Latar Belakang

Pengembangan perangkat lunak, termasuk pembuatan halaman web sederhana, melibatkan perubahan berulang pada berkas seperti tata letak, teks, dan warna. Tanpa pencatatan yang baik, pengembang dapat kesulitan mengetahui versi terakhir serta perubahan yang telah dilakukan. Permasalahan tersebut dapat diatasi dengan sistem pengelolaan versi (version control system), yaitu perangkat lunak yang mencatat perubahan berkas dan menyimpan versi sebelumnya (Blischak et al., 2016).

Salah satu sistem pengelolaan versi yang banyak digunakan adalah Git. Git bersifat terdistribusi, sehingga setiap salinan repository memiliki riwayat perubahan secara lengkap (Ram, 2013). Sementara itu, GitHub menyediakan layanan penyimpanan repository Git secara daring agar proyek dapat dicadangkan, dibagikan, dan dikerjakan bersama (Blischak et al., 2016). Keduanya saling melengkapi dan menjadi bekal penting dalam pengembangan perangkat lunak.

Dalam praktik ini, Git dan GitHub diterapkan pada proyek website Jelajah Nusantara, yaitu halaman informasi wisata sederhana yang dibuat menggunakan HTML dan CSS. Praktik dimulai dari konfigurasi Git, pembuatan repository lokal, pencatatan perubahan melalui commit, hingga menghubungkan repository dengan GitHub. Selanjutnya, dilakukan pengunggahan proyek menggunakan git push dan pengambilan perubahan dari GitHub menggunakan git pull. Seluruh proses tersebut didokumentasikan dalam laporan ini melalui penjelasan dan tangkapan layar sebagai bukti pelaksanaan praktik.

1.2 Fokus Permasalahan

Berdasarkan latar belakang di atas, praktik ini berfokus pada pertanyaan berikut.

1. Apa peran Git dan GitHub serta konsep dasar yang perlu dipahami sebelum menggunakannya?

2. Bagaimana membuat repository lokal dan menyimpan perubahan pada website Jelajah Nusantara dengan perintah `git init`, `git add`, dan `git commit`?
3. Bagaimana menghubungkan repository lokal dengan repository GitHub, mengatur branch utama, dan mengunggah proyek?
4. Bagaimana perubahan yang dibuat langsung di GitHub dapat diambil kembali ke repository lokal?
5. Bukti apa yang terlihat pada terminal dan halaman GitHub bahwa setiap tahap berhasil dijalankan?

1.3 Tujuan Praktik

Praktik ini dilaksanakan dengan tujuan sebagai berikut.

1. Menjelaskan fungsi Git dan GitHub serta istilah-istilah pentingnya.
2. Membuat repository lokal dan merekam perubahan proyek menggunakan `git init`, `git add`, dan `git commit`.
3. Menghubungkan repository lokal dengan GitHub menggunakan `git remote`, mengatur branch `main`, dan mengunggah proyek dengan `git push`.
4. Menambahkan berkas langsung melalui GitHub, lalu mengambilnya ke repository lokal dengan `git pull`.
5. Mendokumentasikan seluruh tahapan secara runtut sebagai laporan praktik.

1.4 Manfaat Praktik

Bagi penulis, praktik ini memberi pengalaman langsung mengelola riwayat proyek, sesuatu yang sulit dipahami hanya dari teori. Penulis juga terbiasa membaca keluaran terminal, misalnya status berkas atau kode commit, untuk memastikan setiap perintah bekerja sesuai harapan. Bagi pembaca, khususnya mahasiswa lain, laporan ini dapat menjadi contoh alur kerja Git dan GitHub dari awal sampai akhir. Adapun bagi pengembangan proyek ke depan, repository yang sudah terbentuk dapat langsung dipakai untuk menambah fitur, membuat branch baru, atau berkolaborasi.

BAB II KONSEP DASAR PENGELOLAAN VERSI

2.1 Pengelolaan Versi dan Alasan Penggunaannya

Banyak orang pernah menyimpan beberapa salinan berkas dengan nama seperti *laporan_final*, *laporan_final_revisi*, lalu *laporan_final_revisi2*. Cara ini cepat menjadi kacau karena tidak jelas apa yang berbeda antarsalinan, siapa yang mengubah, dan kapan. Pengelolaan versi menggantikan kebiasaan tersebut dengan pencatatan yang sistematis: setiap perubahan yang dianggap penting disimpan sebagai satu titik versi lengkap dengan keterangan, penulis, dan waktunya (Blischak et al., 2016).

Manfaat praktisnya cukup jelas. Apabila suatu perubahan ternyata menimbulkan kesalahan, pengguna dapat menelusuri riwayat dan mengembalikan berkas ke kondisi sebelumnya tanpa mengganggu bagian proyek yang lain (Ram, 2013). Riwayat yang tersimpan juga memudahkan pembandingan antarversi dan memperjelas tanggung jawab atas setiap perubahan.

2.2 Git sebagai Sistem Pengelolaan Versi Terdistribusi

Git menyimpan kondisi proyek sebagai rangkaian *snapshot*, yaitu potret seluruh berkas pada saat tertentu, dan bukan sekadar daftar selisih per berkas (Chacon & Straub, 2014). Sebagian besar operasi Git berjalan di komputer lokal, sehingga pengguna dapat mencatat riwayat tanpa harus tersambung ke internet. Karena bersifat terdistribusi, setiap salinan repository memuat riwayat lengkap proyek (Ram, 2013).

Git digunakan melalui antarmuka baris perintah, misalnya terminal pada Visual Studio Code. Perintah-perintahnya pendek dan keluarannya informatif, sehingga pengguna dapat mengecek kondisi repository kapan saja dengan `git status`. Identitas pengguna, yaitu nama dan alamat email, diatur melalui `git config` dan akan melekat pada setiap commit yang dibuat (Git, n.d.-d).

2.3 GitHub sebagai Layanan Hosting Repository

Git dan GitHub sering tertukar, padahal keduanya berbeda. Git adalah alat pencatat versi yang berjalan di komputer, sedangkan GitHub adalah layanan web tempat repository Git disimpan secara daring (Blischak et al., 2016). Dengan

analogi sederhana, Git ibarat buku catatan perubahan di meja kerja, sedangkan GitHub adalah lemari arsip di internet yang dapat dibuka dari mana saja.

GitHub menampilkan isi repository melalui peramban, termasuk daftar berkas, riwayat commit, dan nama pengunggahnya. Berkas `README.md` yang ada di repository otomatis ditampilkan pada halaman depan sebagai deskripsi proyek (GitHub, n.d.-a). GitHub juga memungkinkan berkas dibuat atau disunting langsung dari peramban lalu disimpan sebagai commit (GitHub, n.d.-b), fitur yang dimanfaatkan pada salah satu tahap praktik ini.

2.4 Repository, Staging Area, dan Commit

Repository adalah folder proyek beserta seluruh riwayat perubahannya. Ketika `git init` dijalankan, Git membuat direktori tersembunyi bernama `.git` di dalam folder tersebut sebagai tempat menyimpan riwayat (Git, n.d.-e). Berkas di folder proyek kemudian melewati beberapa keadaan: belum dilacak (*untracked*) atau berubah, siap direkam (*staged*), dan akhirnya tersimpan (*committed*) (Chacon & Straub, 2014).

Staging area dapat dibayangkan sebagai keranjang tempat pengguna memilih perubahan mana yang akan masuk ke versi berikutnya. Perintah `git add` memasukkan berkas ke keranjang ini (Git, n.d.-a). Setelah itu, `git commit` merekam isi keranjang menjadi satu titik riwayat yang disertai pesan penjelasan (Git, n.d.-c). Setiap commit memiliki kode identitas unik, dan tujuh karakter pertamanya biasa ditampilkan sebagai penanda singkat, seperti yang terlihat pada hasil praktik di Bab IV.

2.5 Branch

Branch adalah jalur pengembangan dalam repository. Pengguna dapat membuat branch terpisah untuk mencoba fitur tanpa memengaruhi jalur utama, lalu menggabungkannya kembali bila hasilnya baik. Pada praktik ini hanya satu branch yang digunakan. Git membuat branch awal bernama `master` (lihat Gambar 4.6), sedangkan repository di GitHub memakai nama `main`, sehingga nama branch lokal diselaraskan dengan perintah `git branch -M main` (Git, n.d.-b).

2.6 Remote, Push, dan Pull

Remote adalah versi repository yang disimpan di server, misalnya di GitHub (Blischak et al., 2016). Secara kebiasaan, remote pertama diberi nama `origin` dan didaftarkan dengan `git remote add` (Git, n.d.-h). Setelah terdaftar, `git push` mengirim commit dari repository lokal ke remote. Opsi `-u` pada `git push` menyimpan hubungan antara branch lokal dan branch remote, sehingga perintah `git push` atau `git pull` berikutnya tidak perlu lagi diikuti nama remote dan branch (Git, n.d.-g).

Arah sebaliknya ditangani `git pull`, yang bekerja dengan mengambil (*fetch*) perubahan dari remote lalu mengintegrasikannya ke branch yang sedang aktif (Git, n.d.-f). Bila branch lokal hanya tertinggal dan tidak memiliki commit baru yang berbeda, Git cukup memajukan penunjuk branch ke commit terbaru. Proses ini disebut *fast-forward* dan tidak membuat commit penggabungan. Catatan tentang kondisi remote yang terakhir diketahui Git disimpan secara lokal sebagai *remote-tracking branch*, misalnya `origin/main` (Git, n.d.-i).

2.7 Perintah Git yang Digunakan dalam Praktik

Ringkasan perintah yang dipakai selama praktik disajikan pada Tabel 2.1. Kolom terakhir menunjukkan nomor tahapan praktik (lihat Subbab 3.4) tempat perintah tersebut digunakan.

Perintah	Fungsi	Tahap
`git --version`	Menampilkan versi Git yang terpasang	1
`git config --global`	Mengatur dan melihat identitas pengguna	2
`git init`	Membuat repository Git baru pada folder proyek	6
`git status`	Melihat kondisi berkas dan branch	6,7,8,16
`git add`	Memasukkan berkas ke staging area	7, 14
`git commit -m`	Merekam isi staging area sebagai commit beserta pesan	8, 14

Perintah	Fungsi	Tahap
`git log --oneline`	Menampilkan riwayat commit dalam satu baris per commit	8
`git remote add origin`, `git remote -v`	Mendaftarkan dan memeriksa alamat repository remote	10
`git branch -M main`, `git branch`	Mengganti nama branch dan menampilkan daftar branch	11
`git push -u origin main`, `git push`	Mengirim commit ke GitHub	12, 14
`git pull origin main`	Mengambil dan mengintegrasikan perubahan dari GitHub	16

Tabel 2.1 *Perintah Git yang digunakan dalam praktik*

BAB III PERSIAPAN DAN ALUR PELAKSANAAN

3.1 Waktu dan Tempat Pelaksanaan

Praktik penggunaan Git dan GitHub dilaksanakan dengan keterangan berikut.

Hari/Tanggal : senin, 26 oktober 2026

Tempat : Universitas Nahdlatul Ulama Indonesia

Mata Kuliah : Pengolahan citra digital

Dosen Pengampu : Andi Aljabar, S.Kom.,MTA.

Seluruh tahapan dikerjakan secara mandiri di komputer penulis menggunakan terminal pada Visual Studio Code. Pekerjaan dilakukan pada folder proyek `praktik-github`, sedangkan repository daring dibuat melalui situs GitHub.

3.2 Perangkat dan Aplikasi

Perangkat dan aplikasi yang digunakan tercantum pada Tabel 3.1.

Tabel 3.1 *Perangkat dan aplikasi yang digunakan*

Komponen	Keterangan	Fungsi
Laptop	Lenovo IdeaPad Slim 3	Perangkat utama praktik
Sistem operasi	Windows	Lingkungan menjalankan Git
Git	Versi 2.52.0.windows.1	Pengelolaan versi di komputer lokal
Visual Studio Code	Editor kode beserta terminal terintegrasi	Menulis kode dan menjalankan perintah Git
Peramban web	Google Chrome	Menguji website dan mengakses GitHub
GitHub	Akun `shela04rahma-hash`	Penyimpanan repository daring

3.3 Objek Praktik: Website Jelajah Nusantara

Objek praktik adalah website Jelajah Nusantara yang dibuat dengan HTML dan CSS. Berkas `index.html` memuat struktur dan isi halaman, sedangkan `style.css` mengatur tampilannya. Pada tahap lanjutan ditambahkan dua berkas pendukung, yaitu `README.md` sebagai dokumentasi proyek dan `catatan-`

`praktik.txt` sebagai catatan singkat yang dibuat langsung melalui GitHub. Identitas proyek dan repository dirangkum pada Tabel 3.2.

Tabel 3.2 *Informasi proyek dan repository*

Komponen	Keterangan
Nama proyek	Jelajah Nusantara
Folder proyek lokal	`praktik-github`
Berkas utama	`index.html` dan `style.css`
Berkas tambahan	`README.md` dan `catatan-praktik.txt`
Nama repository	`praktik-github-shela`
Username GitHub	`shela04rahma-hash`
Branch utama	`main`
URL repository	https://github.com/shela04rahma-hash/praktik-github-shela

3.4 Alur Pelaksanaan Praktik

Praktik dijalankan dalam enam belas tahap yang berurutan. Tahap-tahap tersebut dapat dikelompokkan menjadi empat bagian besar.

1. **Penyiapan lingkungan (tahap 1 dan 2).** Memeriksa instalasi Git dan mengatur identitas pengguna.
2. **Pembuatan website (tahap 3 sampai 5).** Menyusun struktur halaman dengan HTML, mengatur tampilan dengan CSS, dan mengujinya melalui peramban.
3. **Pengelolaan versi lokal (tahap 6 sampai 8).** Membuat repository dengan `git init`, memasukkan berkas ke staging area dengan `git add`, dan menyimpannya dengan `git commit`.
4. **Penghubungan dan sinkronisasi dengan GitHub (tahap 9 sampai 16).** Membuat repository di GitHub, menghubungkannya dengan `git remote`, mengatur branch `main`, mengunggah dengan `git push`, memeriksa hasilnya, menambahkan `README.md` dan `catatan-praktik.txt`, lalu mengambil perubahan dengan `git pull`.

Rincian setiap tahap beserta hasil yang terlihat pada tangkapan layar diuraikan pada Bab IV.

BAB IV IMPLEMENTASI DAN HASIL PRAKTIK

4.1 Gambaran Umum Pelaksanaan

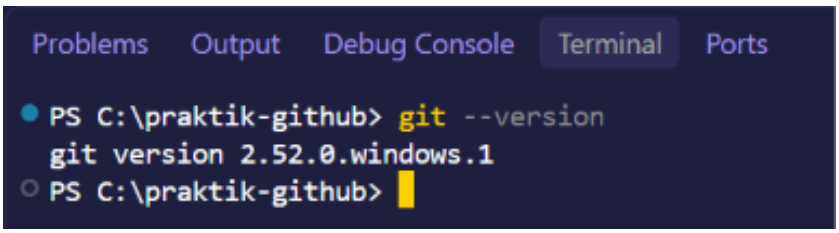
Bab ini menguraikan enam belas tahap praktik sesuai urutan pengerjaan. Setiap tahap dijelaskan melalui tujuan, proses atau perintah yang dijalankan, dan hasil yang tampak pada tangkapan layar. Seluruh perintah dijalankan pada folder `praktik-github` di drive C, sebagaimana terlihat pada baris perintah (*prompt*) di terminal. Rekapitulasi seluruh tahap disajikan pada akhir bab (Tabel 4.1).

4.2 Penyiapan Lingkungan Kerja Git

Bagian ini menjelaskan tahapan awal dalam menyiapkan lingkungan kerja sebelum melakukan pengelolaan proyek menggunakan Git. Persiapan dilakukan untuk memastikan Git telah terpasang, konfigurasi identitas pengguna telah diatur, serta folder proyek telah siap digunakan sebagai repository lokal. Tahapan ini menjadi dasar sebelum melanjutkan proses pencatatan perubahan dan menghubungkan repository dengan GitHub.

4.2.1 Tahap 1: Memeriksa Instalasi Git

Sebelum memulai, penulis memastikan Git sudah terpasang dan dikenali oleh terminal. Pemeriksaan dilakukan dengan menjalankan perintah `git --version`. Terminal membalas dengan `git version 2.52.0.windows.1` (Gambar 4.1). Keluaran tersebut menandakan Git telah terpasang dengan benar di Windows dan perintahnya dapat dipanggil dari terminal. Tampilan panel *Problems*, *Output*, *Debug Console*, *Terminal*, dan *Ports* pada gambar menunjukkan bahwa perintah dijalankan di terminal terintegrasi Visual Studio Code.



Gambar 4.1 *Pemeriksaan versi Git dengan perintah git --version*

4.2.2 Tahap 2: Mengatur Identitas Pengguna Git

Git mencantumkan nama dan alamat email pembuat pada setiap commit, sehingga identitas ini perlu diatur lebih dulu. Penulis menjalankan `git config --global user.name "Shela Rahma Fitri"` dan `git config --global user.email` yang diikuti alamat email pengguna. Opsi `--global` membuat pengaturan berlaku untuk semua repository di komputer tersebut (Git, n.d.-d). Untuk memastikan pengaturan tersimpan, penulis menjalankan `git config --global --list`. Hasilnya pada Gambar 4.2 memperlihatkan dua entri, yaitu `user.name` dan `user.email`, yang nilainya sesuai dengan data yang dimasukkan.



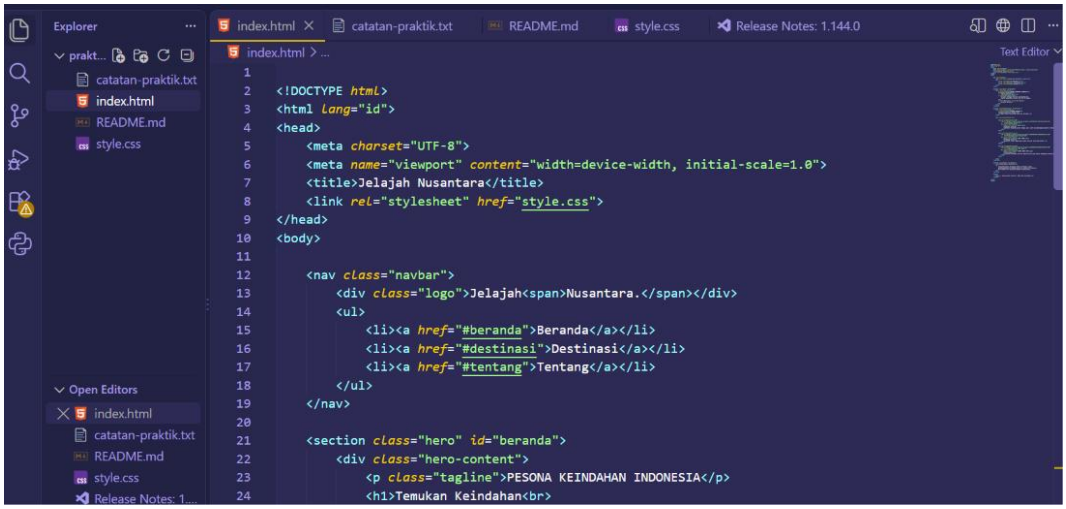
Gambar 4.2 *Pengaturan dan pemeriksaan identitas pengguna Git*

4.3 Pembuatan Website Jelajah Nusantara

Tiga tahap pada bagian ini berkaitan dengan pembuatan website Jelajah Nusantara menggunakan HTML dan CSS, serta pengujian tampilan melalui peramban. Setiap tahap dijelaskan berdasarkan proses pengerjaan dan hasil yang diperoleh. Bukti kegiatan ditampilkan melalui tangkapan layar kode program dan tampilan website.

4.3.1 Tahap 3: Menyusun Struktur Halaman dengan HTML

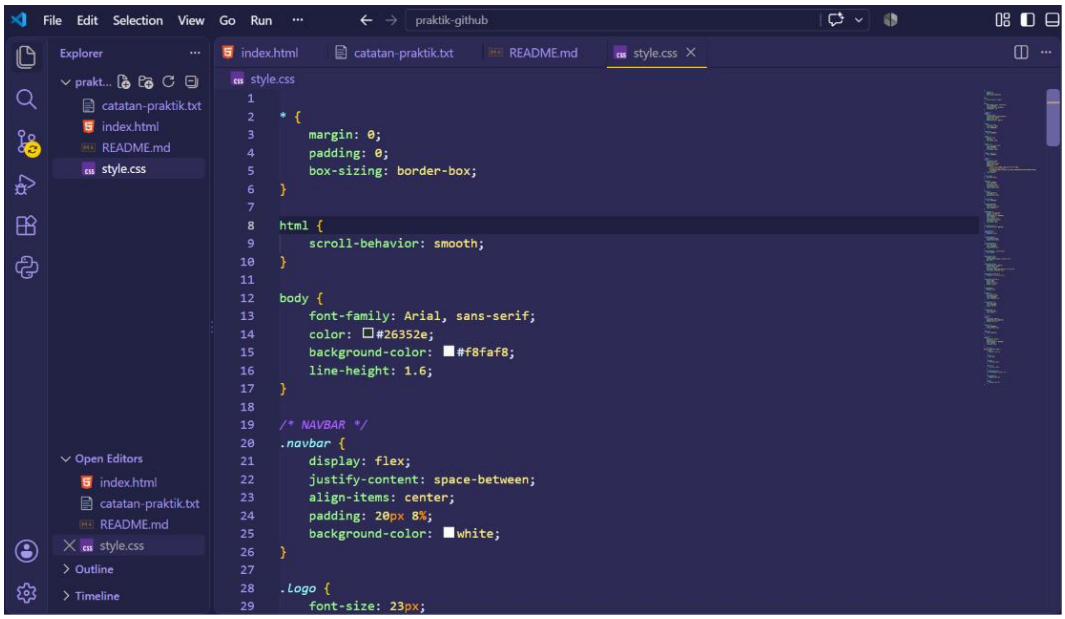
Tahap ini bertujuan membuat kerangka halaman website. HTML dipakai untuk menentukan elemen apa saja yang tampil, seperti judul, teks, dan bagian-bagian halaman. Seluruh struktur ditulis pada berkas `index.html` di folder `praktik-github`. Bagian halaman yang dibuat: Halaman website terdiri dari bagian header, navigasi, konten utama, informasi destinasi wisata, dan footer.



Gambar 4.3 *Struktur halaman website Jelajah Nusantara pada index.html*

4.3.2 Tahap 4: Mengatur Tampilan dengan CSS

Setelah struktur siap, tampilan halaman diatur menggunakan CSS pada berkas `style.css`. Berkas ini mengatur aspek visual seperti warna, jenis dan ukuran huruf, serta penataan elemen. Pemisahan HTML dan CSS membuat isi dan tampilan dapat diubah secara terpisah, dan pada praktik ini keduanya tercatat sebagai dua berkas dalam repository. CSS digunakan untuk mengatur warna, ukuran huruf, dan tata letak elemen agar tampilan website lebih rapi dan menarik.

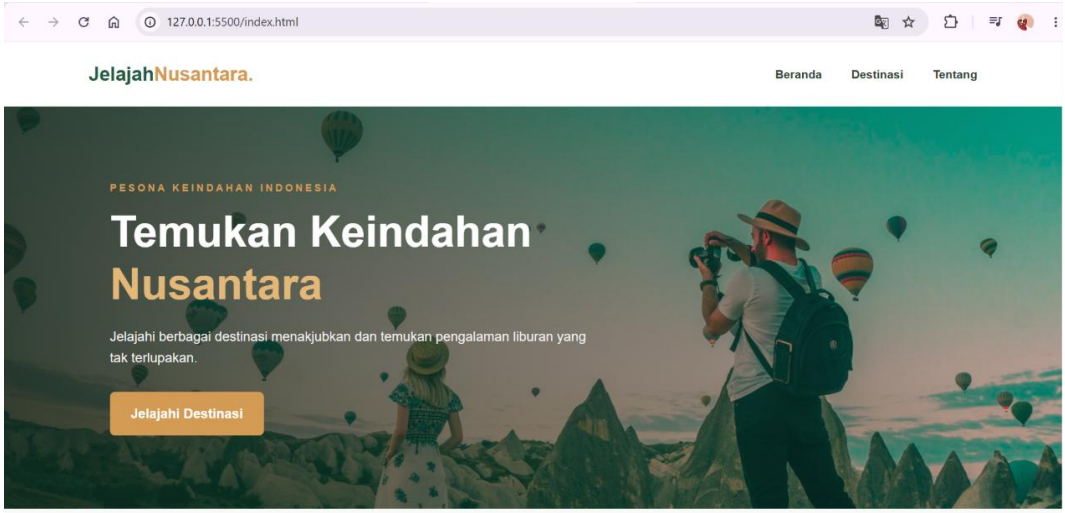


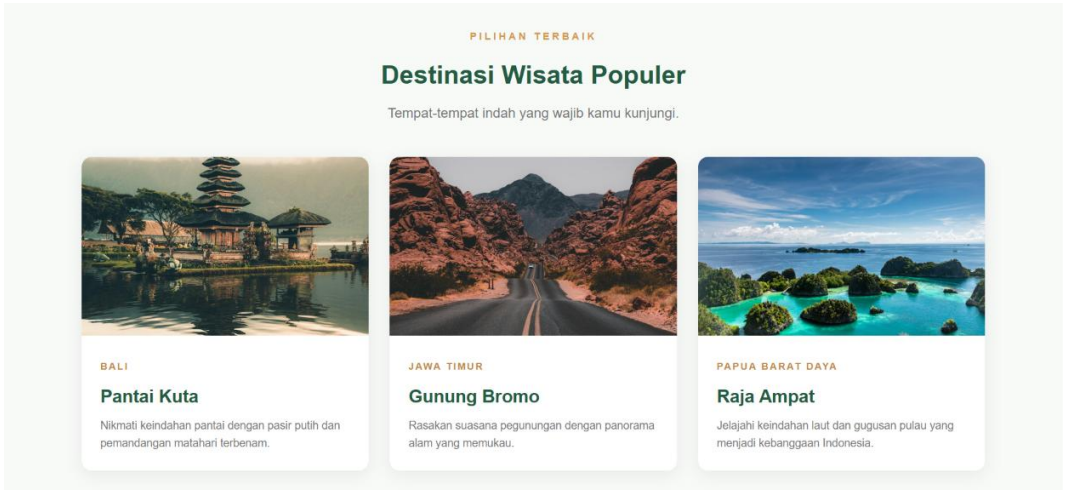
Gambar 4.4 *Pengaturan tampilan website pada style.css*

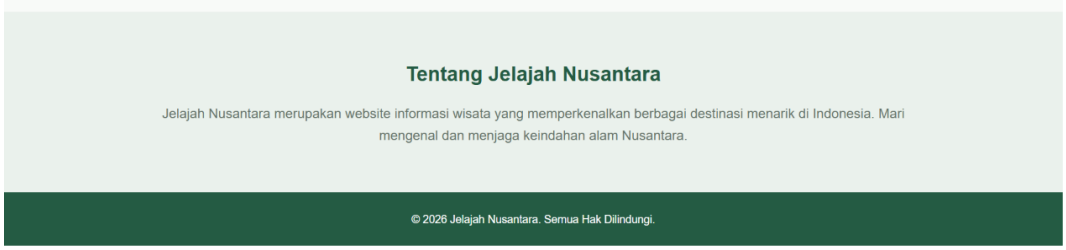
4.3.3 Tahap 5: Menguji Website melalui Peramban

Pengujian dilakukan agar website dipastikan tampil sesuai rancangan sebelum dimasukkan ke repository. Penulis membuka halaman melalui peramban dengan cara: Penulis membuka file index.html menggunakan fitur Live Server pada Visual Studio Code. Website kemudian ditampilkan melalui peramban Google Chrome.

Hasil pengujian: Hasil pengujian menunjukkan bahwa website Jelajah Nusantara berhasil ditampilkan di Google Chrome. Tampilan halaman muncul sesuai dengan pengaturan HTML dan CSS yang telah dibuat.







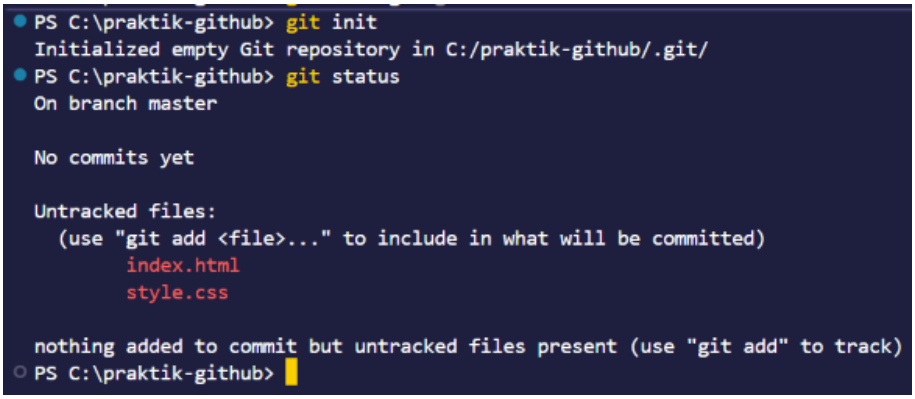
Gambar 4.5 *Tampilan website Jelajah Nusantara pada peramban*

4.4 Pengelolaan Versi pada Repository Lokal

Setelah repository lokal berhasil dibuat, tahap selanjutnya adalah melakukan pengelolaan versi pada proyek website Jelajah Nusantara. Pada tahap ini, file proyek mulai dikelola menggunakan Git melalui proses penambahan file ke staging area, pencatatan perubahan dalam commit, serta pemeriksaan riwayat perubahan. Proses ini bertujuan agar setiap perubahan pada proyek dapat tercatat dan dikelola dengan lebih terstruktur.

4.4.1 Tahap 6: Membuat Repository Lokal dengan git init

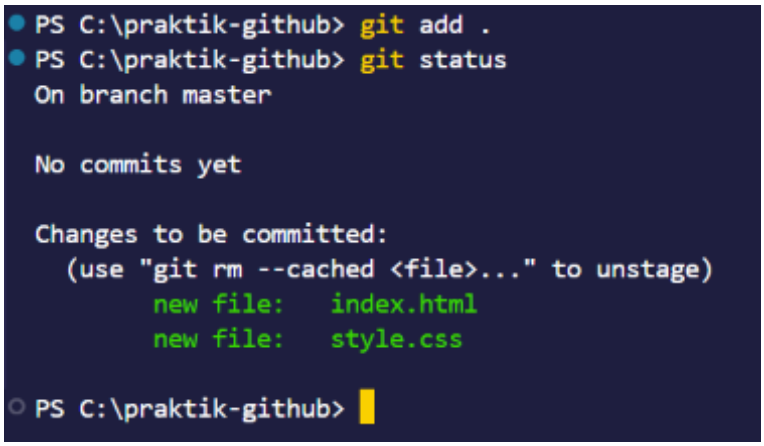
Folder `praktik-github` pada awalnya hanyalah folder biasa. Agar riwayat perubahannya dapat dicatat, penulis menjalankan `git init`. Terminal menjawab `Initialized empty Git repository in C:/praktik-github/.git/` (Gambar 4.6), yang berarti direktori `.git` telah dibuat dan folder tersebut kini menjadi repository. Perintah `git status` kemudian menunjukkan `On branch master` dan `No commits yet`, sedangkan `index.html` dan `style.css` muncul berwarna merah di bawah `Untracked files`. Dengan kata lain, Git sudah mengenali kedua berkas, tetapi belum melacaknya.



Gambar 4.6 *Inisialisasi repository dengan git init dan pemeriksaan git status*

4.4.2 Tahap 7: Memasukkan Berkas ke Staging Area dengan git add

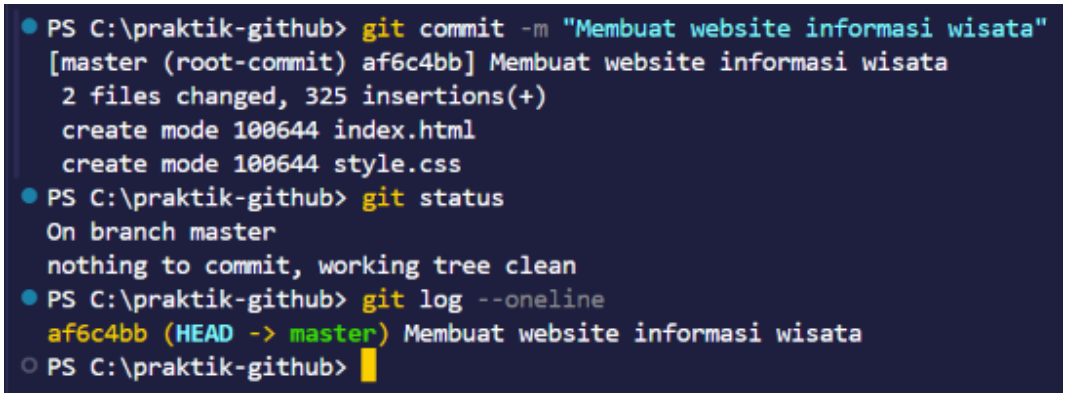
Berkas yang belum dilacak perlu dimasukkan ke staging area terlebih dahulu. Penulis menjalankan `git add .`, di mana tanda titik berarti seluruh perubahan di folder saat ini. Pemeriksaan ulang dengan `git status` pada Gambar 4.7 memperlihatkan perubahan pada bagian `Changes to be committed`: `index.html` dan `style.css` kini berwarna hijau dengan keterangan `new file`. Perubahan warna dari merah ke hijau ini menjadi penanda bahwa kedua berkas sudah siap direkam.



Gambar 4.7 *Hasil git status setelah git add . dijalankan*

4.4.3 Tahap 8: Menyimpan Perubahan dengan git commit

Tahap berikutnya merekam isi staging area menjadi commit pertama dengan perintah `git commit -m "Membuat website informasi wisata"`. Keluaran pada Gambar 4.8 menunjukkan `[master (root-commit) af6c4bb]`, artinya ini adalah commit paling awal di repository dengan kode `af6c4bb`. Terminal juga melaporkan dua berkas berubah dengan 325 baris penambahan, lengkap dengan `create mode 100644` untuk `index.html` dan `style.css`. Setelah itu, `git status` menampilkan `nothing to commit, working tree clean`, dan `git log --oneline` memperlihatkan satu baris riwayat berupa `af6c4bb (HEAD -> master) Membuat website informasi wisata`. Hasil ini membuktikan bahwa versi pertama website sudah tercatat di repository lokal.



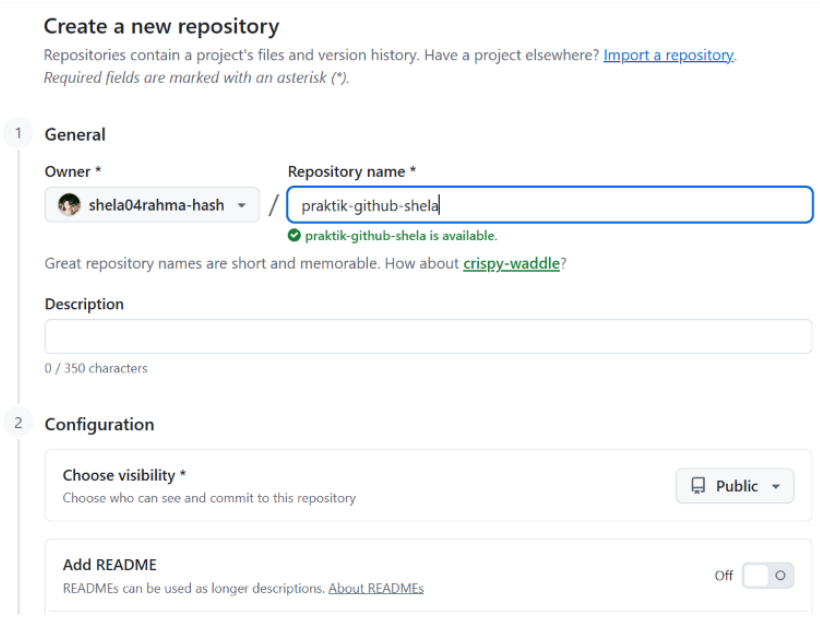
Gambar 4.8 *Commit pertama beserta pemeriksaan git status dan git log --oneline*

4.5 Penghubungan Repository Lokal dengan GitHub

Setelah proses pengelolaan versi pada repository lokal selesai dilakukan, tahap berikutnya adalah menghubungkan repository lokal dengan repository yang telah dibuat di GitHub. Proses ini bertujuan agar proyek yang tersimpan di komputer dapat terhubung dengan penyimpanan online. Pada tahap ini dilakukan penambahan alamat repository remote, penyesuaian nama branch menjadi main, serta pengiriman file proyek ke GitHub menggunakan perintah push.

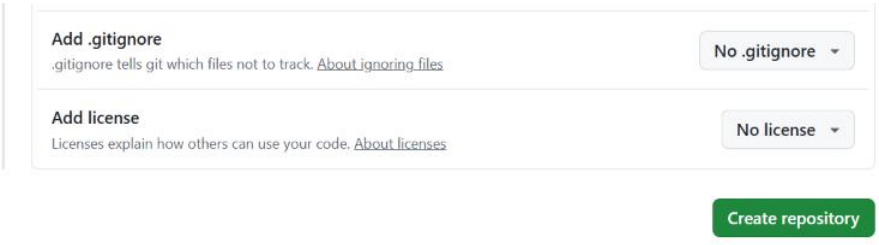
4.5.1 Tahap 9: Membuat Repository Baru di GitHub

Repository lokal membutuhkan pasangan di GitHub sebagai tempat penyimpanan daring. Pembuatannya dilakukan melalui halaman *Create a new repository*. Pada Gambar 4.9, kolom *Owner* berisi akun `shela04rahma-hash` dan kolom *Repository name* diisi `praktik-github-shela`, disertai keterangan bahwa nama tersebut tersedia (*is available*). Kolom deskripsi dibiarkan kosong, dan visibilitas dipilih *Public*. Opsi *Add README* berada pada posisi *Off*.



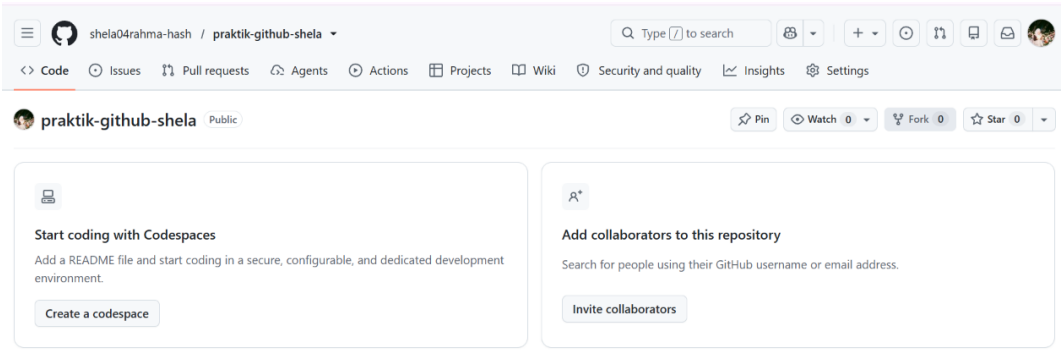
Gambar 4.9 *Formulir pembuatan repository baru pada GitHub*

Bagian bawah formulir (Gambar 4.10) memperlihatkan `.gitignore` dan lisensi sama-sama tidak dipilih (*No .gitignore* dan *No license*), lalu proses diselesaikan dengan menekan tombol *Create repository*. Dengan tidak menambahkan berkas awal apa pun, repository di GitHub dibiarkan kosong, sehingga commit dari komputer lokal dapat dikirim tanpa harus digabungkan lebih dulu dengan berkas bawaan dari GitHub.



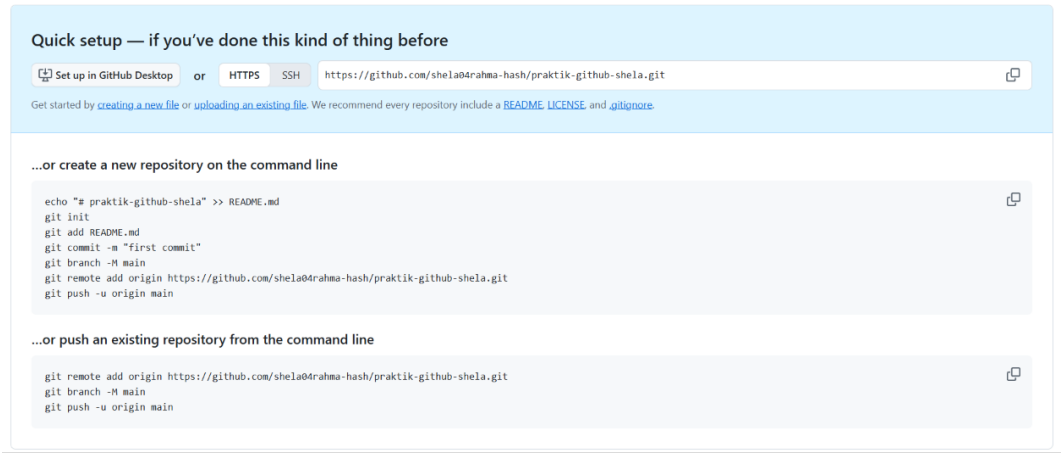
Gambar 4.10 *Pilihan .gitignore dan lisensi serta tombol Create repository*

Setelah repository terbentuk, GitHub menampilkan halaman repository kosong `praktik-github-shela` berstatus *Public* (Gambar 4.11). Halaman ini menawarkan beberapa pintasan, antara lain membuat *codespace* dan mengundang kolaborator, yang tidak digunakan dalam praktik ini.



Gambar 4.11 *Halaman repository praktik-github-shela setelah berhasil dibuat*

Pada halaman yang sama terdapat panduan *Quick setup* (Gambar 4.12) yang memuat alamat HTTPS repository, yaitu `https://github.com/shela04rahma-hash/praktik-github-shela.git`, serta contoh perintah untuk mengunggah repository yang sudah ada: `git remote add origin`, `git branch -M main`, dan `git push -u origin main`. Ketiga perintah inilah yang dijalankan pada tahap 10 sampai 12.



Gambar 4.12 *Panduan Quick setup beserta alamat repository*

4.5.2 Tahap 10: Menghubungkan Repository Lokal dengan git remote

Agar repository lokal mengetahui alamat tujuan pengiriman, penulis menjalankan `git remote add origin` diikuti alamat repository dari GitHub. Nama `origin` dipakai sebagai nama pendek untuk alamat tersebut. Hasilnya diperiksa dengan `git remote -v`, dan Gambar 4.13 menampilkan dua baris `origin`, masing-masing bertanda `(fetch)` dan `(push)`, dengan alamat yang sama.

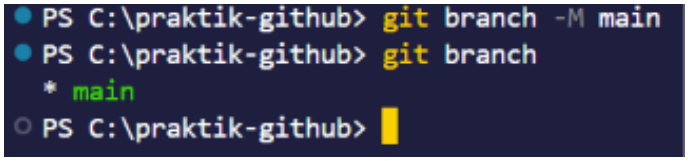
Artinya remote terdaftar untuk dua arah: mengambil perubahan dan mengirim perubahan.



Gambar 4.13 *Penambahan remote origin dan pemeriksaan dengan git remote -v*

4.5.3 Tahap 11: Mengatur Branch Utama menjadi main

Seperti terlihat pada Tahap 6, branch awal yang dibuat Git bernama `master`, sedangkan branch pada repository GitHub bernama `main`. Penulis menyelaraskannya dengan perintah `git branch -M main`, kemudian memeriksa daftar branch dengan `git branch`. Pada Gambar 4.14 terlihat satu entri, yaitu `* main` berwarna hijau, di mana tanda bintang menandakan branch yang sedang aktif. Dengan demikian, nama branch lokal sudah berubah dari `master` menjadi `main`.



Gambar 4.14 *Perubahan nama branch menjadi main dan pemeriksaan dengan git branch*

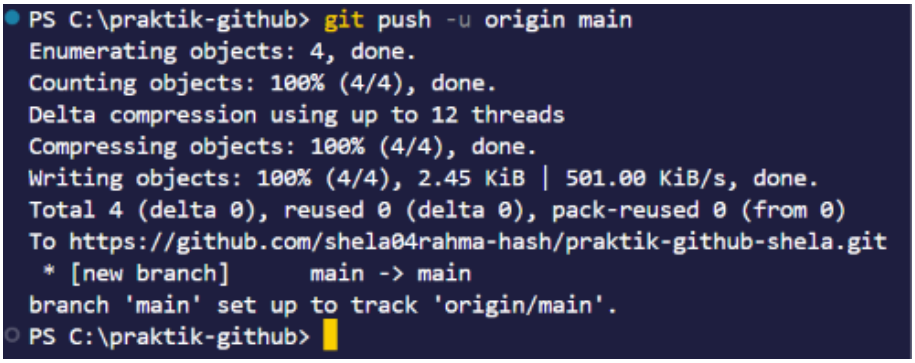
4.6 Sinkronisasi Proyek antara Komputer Lokal dan GitHub

Tahap selanjutnya adalah melakukan sinkronisasi antara repository lokal dan repository yang terdapat di GitHub. Proses ini dilakukan untuk memastikan perubahan yang dibuat pada salah satu repository dapat diperbarui pada repository lainnya. Dalam praktik ini, sinkronisasi dilakukan dengan menambahkan file catatan-praktik.txt secara langsung melalui GitHub, kemudian mengambil perubahan tersebut ke komputer lokal menggunakan perintah git pull origin main. Setelah proses selesai, dilakukan pemeriksaan status repository untuk memastikan bahwa repository lokal telah sesuai dengan repository di GitHub.

4.6.1 Tahap 12: Mengunggah Proyek dengan git push

Setelah remote dan branch siap, proyek dikirim ke GitHub dengan perintah `git push -u origin main`. Terminal menampilkan proses penghitungan dan kompresi objek (*Counting objects: 100% (4/4)* dan *Compressing objects: 100%*

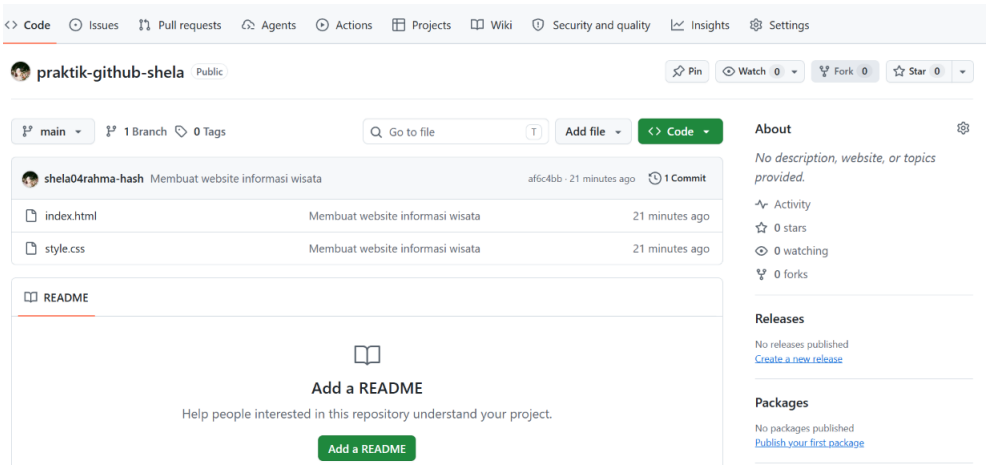
(4/4)), disusul penulisan objek sebesar 2,45 KiB. Baris `* [new branch] main -> main` menandakan branch `main` dibuat di repository GitHub, dan baris terakhir `branch 'main' set up to track 'origin/main'` menandakan opsi `-u` berhasil menghubungkan kedua branch (Gambar 4.15). Hubungan ini yang membuat perintah `git push` pada tahap 14 dapat dijalankan tanpa tambahan argumen.



Gambar 4.15 *Proses pengunggahan proyek dengan git push -u origin main*

4.6.2 Tahap 13: Memeriksa Hasil Pengunggahan di GitHub

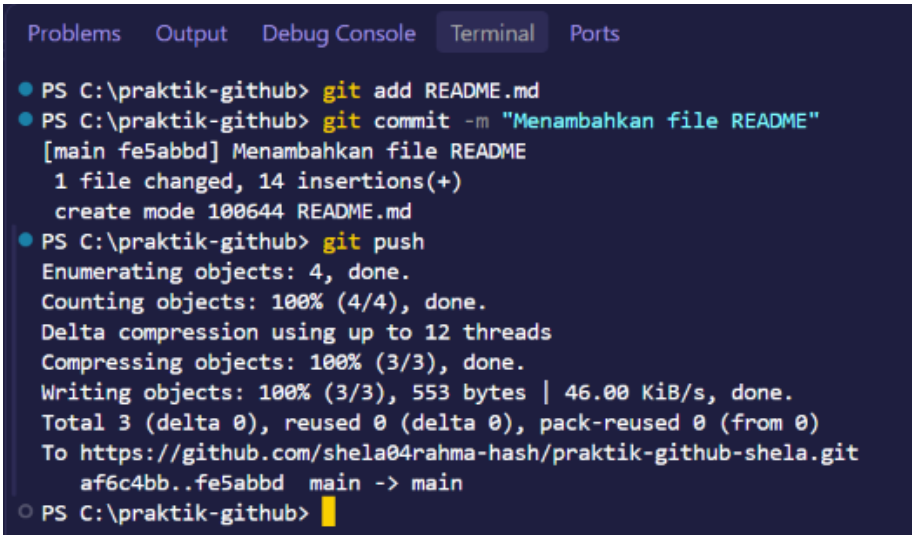
Keberhasilan `git push` perlu dipastikan lewat tampilan di GitHub. Halaman repository pada Gambar 4.16 menunjukkan branch `main` dengan *1 Branch* dan *1 Commit*. Dua berkas, `index.html` dan `style.css`, tampil dengan pesan commit `Membuat website informasi wisata`, dan kode commit pada GitHub, `af6c4bb`, sama persis dengan kode yang tercatat di terminal pada tahap 8. Kesamaan ini menunjukkan bahwa commit yang diunggah adalah commit yang sama. Pada bagian bawah halaman GitHub menampilkan ajakan *Add a README* karena repository belum memiliki berkas `README.md`, yang kemudian ditangani pada tahap berikutnya.



Gambar 4.16 *Tampilan repository GitHub setelah proyek berhasil diunggah*

4.6.3 Tahap 14: Membuat dan Mengunggah README.md

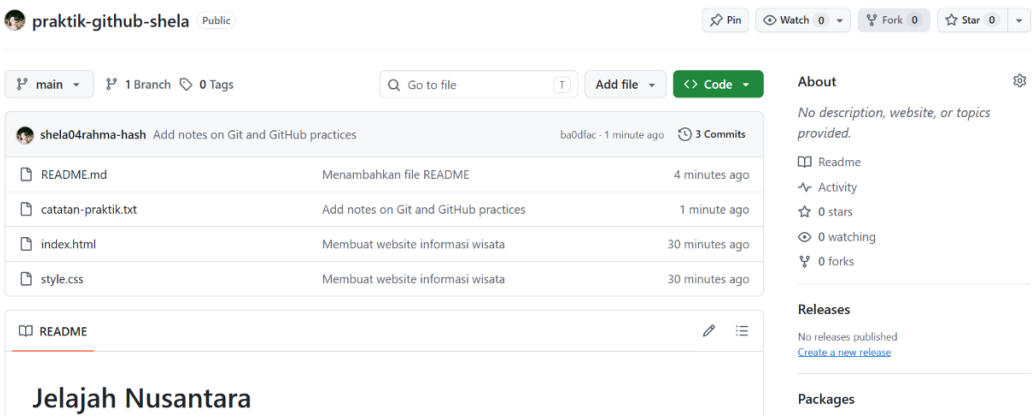
`README.md` berfungsi sebagai dokumentasi proyek dan ditampilkan GitHub pada halaman depan repository (GitHub, n.d.-a). Berkas ini dibuat di komputer lokal, lalu dimasukkan ke staging area dengan `git add README.md`. Penulis menyimpannya dengan `git commit -m "Menambahkan file README"` dan mendapat keluaran `[main fe5abbd] Menambahkan file README`, yaitu commit dengan kode `fe5abbd` yang memuat satu berkas baru dengan 14 baris. Terakhir, perintah `git push` dijalankan tanpa argumen. Baris `af6c4bb..fe5abbd main -> main` pada Gambar 4.17 memperlihatkan bahwa branch `main` di GitHub maju dari commit pertama ke commit `fe5abbd`. Isi `README.md` yang berjudul *Jelajah Nusantara* terlihat pada halaman GitHub di Gambar 4.18.



Gambar 4.17 *Proses menambahkan README.md dengan git add, git commit, dan git push*

4.6.4 Tahap 15: Menambahkan catatan-praktik.txt Langsung melalui GitHub

Tahap ini sengaja dilakukan bukan lewat terminal, melainkan langsung di GitHub, agar terjadi situasi di mana repository daring memiliki perubahan yang belum ada di komputer lokal. Penulis membuat berkas `catatan-praktik.txt` melalui antarmuka web dan menyimpannya sebagai commit. Gambar 4.18 memperlihatkan hasilnya: commit terbaru atas nama `shela04rahma-hash` berpesan `Add notes on Git and GitHub practices` dengan kode `ba0dfac`, dan penghitung riwayat menunjukkan *3 Commits*. Daftar berkas kini berisi `README.md`, `catatan-praktik.txt`, `index.html`, dan `style.css`. Tangkapan layar proses pembuatan berkas di GitHub tidak termasuk dalam bukti yang tersedia, sehingga hanya hasil akhirnya yang ditampilkan.



Gambar 4.18 *Tampilan repository setelah catatan-praktik.txt ditambahkan melalui GitHub*

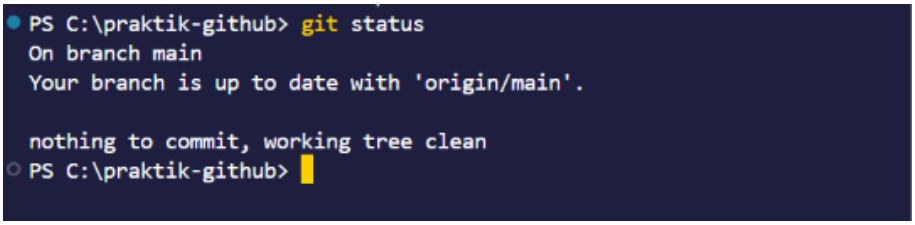
4.6.5 Tahap 16: Mengambil Perubahan dari GitHub dengan git pull

Pada tahap ini, repository lokal belum memiliki file catatan-praktik.txt yang dibuat melalui GitHub. Penulis menjalankan git pull origin main untuk mengambil perubahan tersebut. Hasil pada Gambar 4.19 menunjukkan pembaruan dari commit fe5abbd ke ba0dfac melalui proses *fast-forward*, yaitu pembaruan tanpa membuat commit penggabungan karena tidak ada perubahan lokal yang berbeda. File catatan-praktik.txt berhasil ditambahkan dengan sembilan baris isi (Git, n.d.-f).

```
PS C:\praktik-github> git pull origin main
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 1.20 KiB | 68.00 KiB/s, done.
From https://github.com/shela04rahma-hash/praktik-github-shela
 * branch            main       -> FETCH_HEAD
   fe5abbd..ba0dfac  main       -> origin/main
Updating fe5abbd..ba0dfac
Fast-forward
 catatan-praktik.txt | 9 +++++++++
 1 file changed, 9 insertions(+)
 create mode 100644 catatan-praktik.txt
```

Gambar 4.19 *Pengambilan perubahan dari GitHub dengan git pull origin main*

Sebagai pemeriksaan akhir, `git status` dijalankan kembali. Keluarannya pada Gambar 4.20 menyatakan `On branch main`, `Your branch is up to date with 'origin/main'`, dan `nothing to commit, working tree clean`. Repository lokal sudah sejajar dengan GitHub dan tidak ada perubahan yang tertinggal.



Gambar 4.20 *Hasil git status yang menunjukkan branch main sudah sesuai dengan origin/main*

4.7 Rekapitulasi dan Pembahasan Hasil

Ringkasan keenam belas tahap beserta buktinya :

No	Tahap	Perintah atau Aktivitas	Hasil Utama	Gambar
1	Memeriksa instalasi Git	`git --version`	Versi 2.52.0.windows.1	4.1
2	Mengatur identitas pengguna	`git config --global`	Nama dan email tersimpan	4.2
3	Menyusun halaman dengan HTML	Menulis `index.html`	File index.html berhasil dibuat.	4.3
4	Mengatur tampilan dengan CSS	Menulis `style.css`	File style.css berhasil dibuat untuk mengatur tampilan website.	4.4
5	Menguji di peramban	Membuka website	Website berhasil ditampilkan di browser.	4.5
6	Membuat repository lokal	`git init`, `git status`	Repository dibuat, 2 berkas untracked	4.6
7	Memasukkan ke staging area	`git add .`	2 berkas berstatus new file	4.7
8	Menyimpan perubahan	`git commit -m`	Commit af6c4bb	4.8
9	Membuat repository GitHub	Formulir di situs GitHub	Repository praktik-github-shela	4.9–4.12
10	Menghubungkan repository	`git remote add origin`	Remote origin (fetch dan push)	4.13
11	Mengatur branch utama	`git branch -M main`	Branch aktif main	4.14
12	Mengunggah proyek	`git push -u origin main`	Branch main terhubung ke origin/main	4.15
13	Memeriksa hasil di GitHub	Halaman repository	2 berkas dan 1 commit tampil	4.16

No	Tahap	Perintah atau Aktivitas	Hasil Utama	Gambar
14	Mengunggah README.md	`git add`, `commit`, `push`	Commit fe5abbd	4.17
15	Menambah berkas lewat GitHub	Fitur tambah berkas di GitHub	Commit ba0dfac, 3 commit	4.18
16	Mengambil perubahan	`git pull origin main`	Fast-forward, working tree clean	4.19, 4.20

Tabel 4.1 *Rekapitulasi tahapan praktik*

Dari tabel tersebut terlihat bahwa sebagian besar tahap didukung bukti langsung berupa keluaran terminal atau tampilan GitHub. Beberapa hal layak dicatat dari hasil praktik.

1. **Perjalanan berkas dapat ditelusuri dari satu status ke status berikutnya.** Berkas index.html dan style.css awalnya berstatus *untracked* (Gambar 4.6), kemudian masuk ke *staging area* dengan status *new file* (Gambar 4.7), tersimpan dalam commit (Gambar 4.8), dan akhirnya tampil di GitHub (Gambar 4.16).
2. **Kode commit konsisten antara terminal dan GitHub.** Commit af6c4bb tercatat di terminal dan halaman repository. Selanjutnya, commit fe5abbd mencatat penambahan README.md, sedangkan ba0dfac mencatat penambahan catatan-praktik.txt melalui GitHub. Kedua perubahan tersebut berhasil disinkronkan ke komputer lokal.
3. **Opsi -u menyederhanakan perintah berikutnya.** Setelah menjalankan git push -u origin main, perintah git push pada tahap 14 dapat digunakan tanpa perlu menyebutkan nama remote dan branch kembali.
4. **Dua jalur pengubahan dapat berjalan berdampingan.** Penambahan berkas dilakukan dari komputer lokal pada tahap 14 dan melalui antarmuka GitHub pada tahap 15. Perubahan dari GitHub kemudian berhasil diambil ke komputer lokal menggunakan git pull origin main tanpa kendala.

BAB V PENUTUP

5.1 Kesimpulan

Berdasarkan praktik yang telah dilakukan, penerapan Git dan GitHub pada proyek website Jelajah Nusantara berhasil dilaksanakan melalui enam belas tahapan. Proses dimulai dari konfigurasi Git, pembuatan repository lokal, pencatatan perubahan menggunakan *staging area* dan *commit*, hingga menghubungkan repository lokal dengan GitHub. Seluruh file proyek berhasil diunggah dan perubahan yang dilakukan melalui GitHub juga berhasil disinkronkan ke komputer lokal menggunakan perintah *pull*.

Melalui praktik ini, penulis memahami bahwa Git digunakan untuk mengelola dan mencatat riwayat perubahan proyek, sedangkan GitHub berfungsi sebagai tempat penyimpanan repository secara daring. Hasil akhir menunjukkan bahwa repository lokal dan GitHub berhasil disinkronkan dengan kondisi *working tree* bersih (*clean*). Dengan demikian, tujuan praktik untuk memahami penggunaan dasar Git dan GitHub telah tercapai.

5.2 Saran

Berdasarkan praktik yang telah dilakukan, penulis menyarankan agar penggunaan Git dan GitHub dapat terus dipelajari dan diterapkan dalam pengembangan proyek berikutnya. Selain memahami perintah dasar seperti *add*, *commit*, *push*, dan *pull*, pengguna juga dapat mempelajari fitur lainnya, seperti penggunaan *branch* untuk mengelola perubahan pada proyek. Dengan latihan secara berkelanjutan, pengelolaan versi proyek diharapkan dapat dilakukan dengan lebih terstruktur, serta mempermudah proses penyimpanan dan sinkronisasi file antara komputer lokal dan GitHub.

DAFTAR PUSTAKA

Blischak, J. D., Davenport, E. R., & Wilson, G. (2016). A quick introduction to version control with Git and GitHub. *PLOS Computational Biology, 12*(1), e1004668. https://doi.org/10.1371/journal.pcbi.1004668

Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. https://git-scm.com/book/en/v2

Git. (n.d.-a). *git-add documentation*. Diakses 31 Oktober 2026, dari https://git-scm.com/docs/git-add

Git. (n.d.-b). *git-branch documentation*. Diakses 31 Oktober 2026, dari https://git-scm.com/docs/git-branch

Git. (n.d.-c). *git-commit documentation*. Diakses 31 Oktober 2026, dari https://git-scm.com/docs/git-commit

Git. (n.d.-d). *git-config documentation*. Diakses 31 Oktober 2026, dari https://git-scm.com/docs/git-config

Git. (n.d.-e). *git-init documentation*. Diakses 31 Oktober 2026, dari https://git-scm.com/docs/git-init

Git. (n.d.-f). *git-pull documentation*. Diakses 31 Oktober 2026, dari https://git-scm.com/docs/git-pull

Git. (n.d.-g). *git-push documentation*. Diakses 31 Oktober 2026, dari https://git-scm.com/docs/git-push

Git. (n.d.-h). *git-remote documentation*. Diakses 31 Oktober 2026, dari https://git-scm.com/docs/git-remote

Git. (n.d.-i). *gitglossary documentation*. Diakses 31 Oktober 2026, dari https://git-scm.com/docs/gitglossary

GitHub. (n.d.-a). *About READMEs*. GitHub Docs. Diakses 31 Oktober 2026, dari https://docs.github.com/en/repositories/managing-your-repository-settings-and-features/customizing-your-repository/about-readmes

GitHub. (n.d.-b). *Quickstart for repositories*. GitHub Docs. Diakses 31 Oktober 2026, dari https://docs.github.com/en/repositories/creating-and-managing-repositories/quickstart-for-repositories

Ram, K. (2013). Git can facilitate greater reproducibility and increased transparency in science. *Source Code for Biology and Medicine, 8*, 7. https://doi.org/10.1186/1751-0473-8-7